



A Comprehensive Measure of Well-Being

Predicting the Human Development Index (HDI) using Machine Learning

Program	Artificial Intelligence and Machine Learning
Project Type	Group Project
Domain	Healthcare / Socio-Economic Development
Team Lead	Pujitha Ulli
Team Members	Abhilash Maguluri, Thota Raja Thota, Maramreddy Naga Lakshmi, Thati Pravallika
GitHub Repository	https://github.com/mlasya927/hdi-project



1. Introduction

The Human Development Index (HDI) is a statistical composite index of life expectancy, education, and per-capita income indicators used to rank countries into four tiers: Very High, High, Medium, and Low human development. A country achieves a higher HDI when its population enjoys a longer lifespan, better educational attainment, and a higher Gross National Income (GNI PPP) per capita.

The HDI was developed to emphasize that people and their capabilities should be the primary measure of a country's development rather than economic growth alone. This project builds a machine learning classification model that predicts a country's HDI category from four core indicators, and exposes it through an interactive Flask web application.

1.1 Problem Scenarios

Scenario	Description	Expected Result
Very High Development	High life expectancy, strong schooling, high GNI per capita	Very High HDI
Emerging Economy	Moderate life expectancy, average education, moderate income	Medium HDI
Needs Intervention	Low life expectancy, limited education, low GNI per capita	Low HDI

2. System Requirements

2.1 Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB free space
- Internet connection for downloading packages and datasets

2.2 Software Requirements

- Operating System: Windows / Linux / macOS
- Python 3.x
- Anaconda Navigator
- Jupyter Notebook or Spyder IDE
- Flask Framework

2.3 Skills & Libraries Used

- Python (Programming Language)
- Anaconda
- Scikit-learn
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Flask (Web Framework)
- Machine Learning

3. Project Methodology (Epics)

The project was executed across 8 epics, taking the data from raw indicators to a fully deployed web application:

Project Workflow — 8 Epics

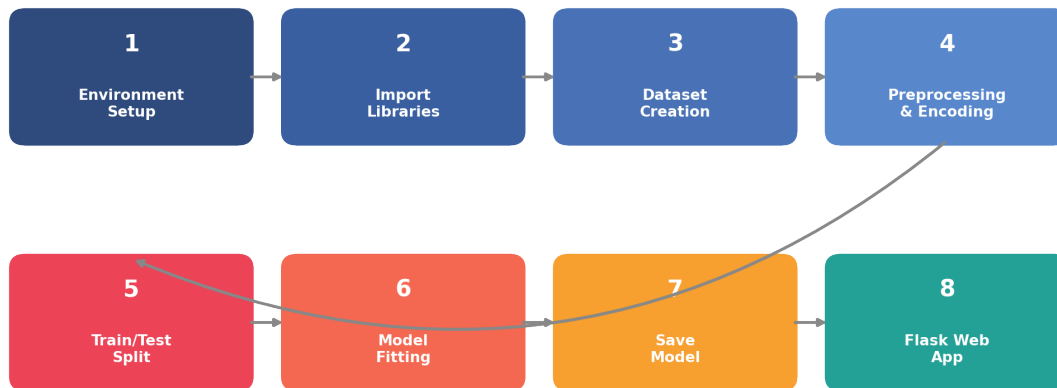


Figure 1: End-to-end project workflow across all 8 epics

#	Epic	Description
1	Environment Setup	Installed Python, Anaconda, and all required packages (requirements.txt).
2	Import Libraries	Imported pandas, numpy, scikit-learn, matplotlib, seaborn, flask.
3	Dataset Creation	Generated a country-level dataset (1,200 records) using UNDP-style HDI methodology.
4	Preprocessing & Encoding	Cleaned data, label-encoded the target, scaled features with StandardScaler.
5	Train/Test Split	Split data 80/20 (960 train / 240 test) using stratified sampling.
6	Model Fitting	Trained a Random Forest Classifier (200 trees, max depth 8, class-balanced).
7	Model Saving	Persisted the trained model, scaler, and label encoder as .pkl files.
8	Flask Web App	Built a form-based web app returning predicted HDI category + confidence.

4. Dataset

A synthetic, UNDP-methodology-based dataset of 1,200 country-level records was generated. Each record contains four development indicators, from which an HDI score (0-1) was computed as the geometric mean of the normalized Life, Education, and Income indices, and then bucketed into four categories.

Feature	Description	Range
Life_Expectancy	Life expectancy at birth (years)	40 - 85
Mean_Years_Schooling	Average years of schooling completed	0 - 15
Expected_Years_Schooling	Expected years of schooling for a child	0 - 20
GNI_per_Capita	Gross National Income per capita (PPP \$)	500 - 90,000
HDI_Category (target)	Very High / High / Medium / Low	4 classes

**Dataset Distribution by HDI Category
(1,200 records)**

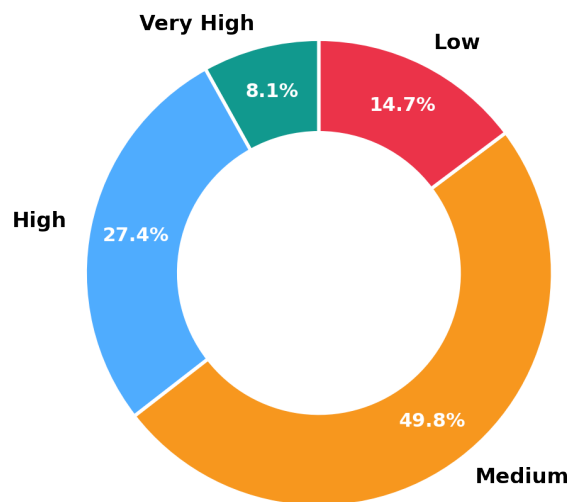


Figure 2: Distribution of the 1,200 records across HDI categories

4.1 Indicator Distributions

Distribution of Development Indicators

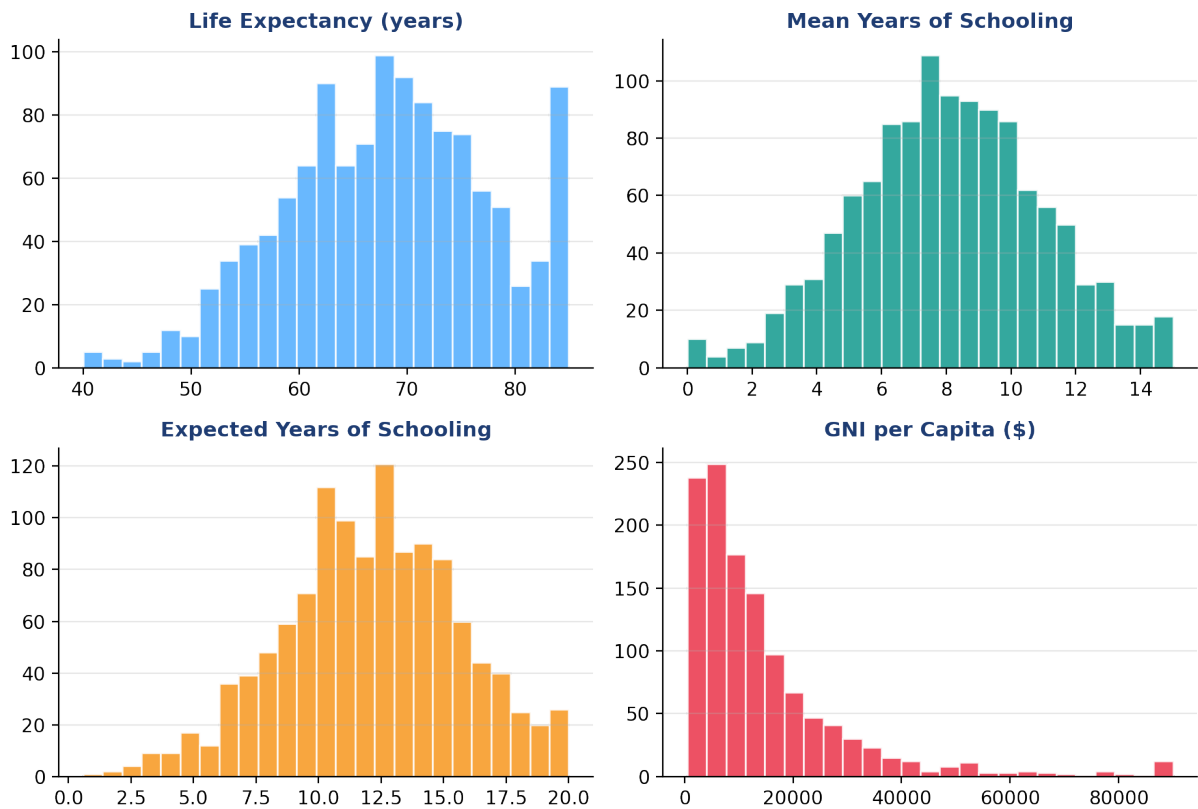


Figure 3: Spread of the four raw development indicators across the dataset

5. Model Building & Evaluation

A **Random Forest Classifier** (`n_estimators=200`, `max_depth=8`, `class_weight='balanced'`) was trained on the scaled features. This ensemble method was chosen for its robustness to feature scale, resistance to overfitting on tabular data, and ability to report feature importance.

Overall Test Accuracy: 86.67%

HDI Category	Precision	Recall	F1-Score	Support
High	0.84	0.94	0.89	66
Low	0.81	0.72	0.76	36
Medium	0.89	0.92	0.90	119
Very High	1.00	0.58	0.73	19

5.1 Confusion Matrix

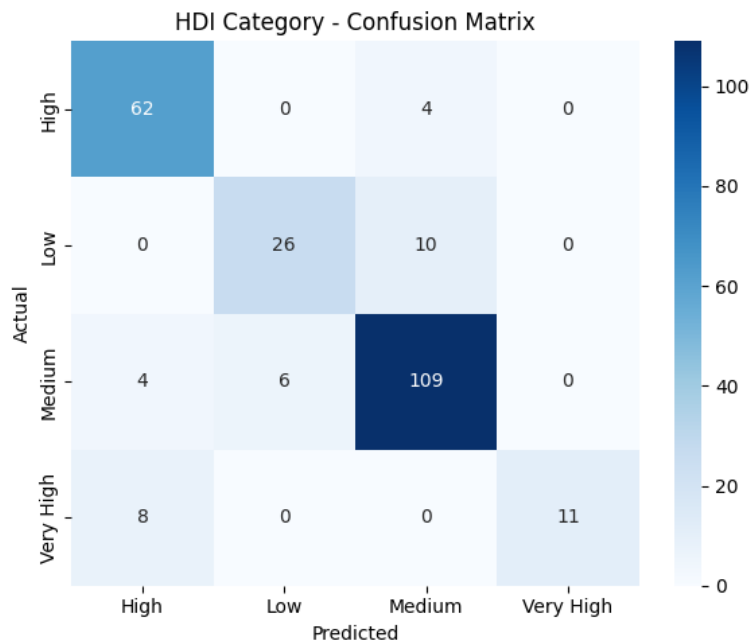


Figure 4: Actual vs. predicted HDI categories on the test set

5.2 Feature Importance

GNI per capita and life expectancy emerged as the strongest predictors of HDI category, followed closely by expected and mean years of schooling.

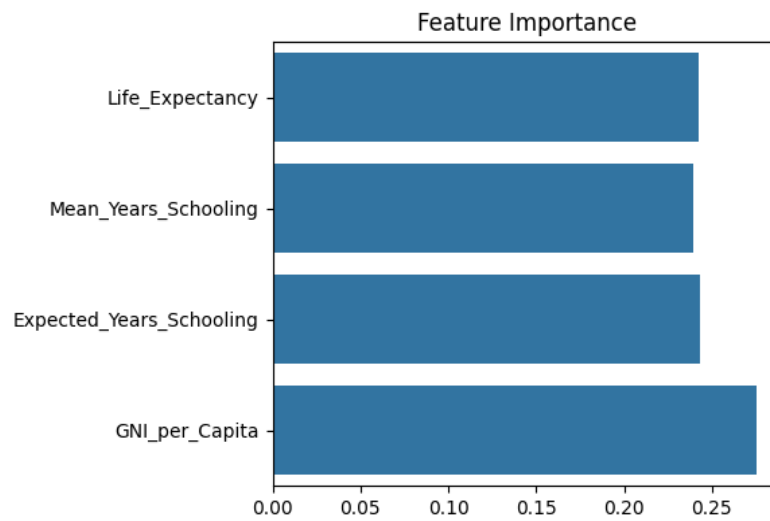


Figure 5: Relative importance of each indicator in the trained model

6. Scenario Validation

The trained model was validated against the three project scenarios to confirm real-world behavior:

Scenario	Life Exp.	Mean School	Expected School	GNI/Capita	Predicted	Confidence
Very High Development	82	13	17	\$55,000	Very High	96.8%
Emerging Economy	68	8	12	\$8,000	Medium	70.8%
Needs Intervention	52	3	6	\$1,200	Low	99.5%

All three scenarios matched the expected HDI category described in the project brief, confirming the model behaves consistently with domain expectations.

7. Flask Web Application

The trained model, scaler, and label encoder are loaded into a Flask application (**app.py**). Users enter the four indicators into a web form; the app scales the input using the same StandardScaler from training, predicts the HDI category, and displays the result along with the model's confidence score.

Flask Application — Prediction Flow

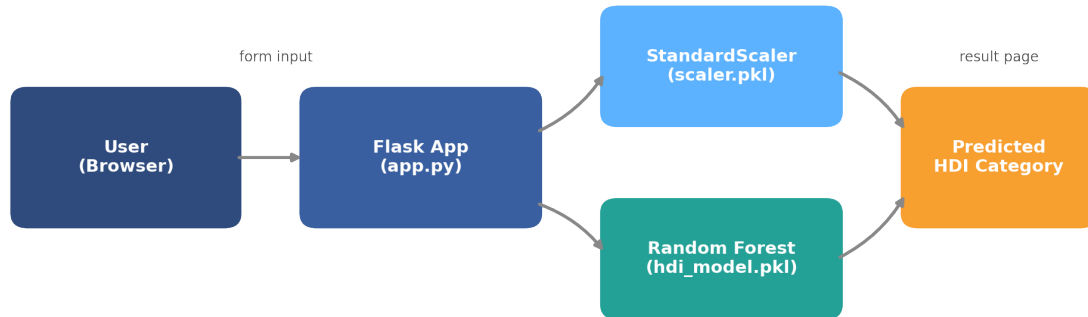


Figure 6: Request flow from browser to model and back

Routes:

- GET / - renders the input form (templates/index.html)
- POST /predict - scales input, runs model.predict(), renders result (templates/result.html)

7.1 Application Preview

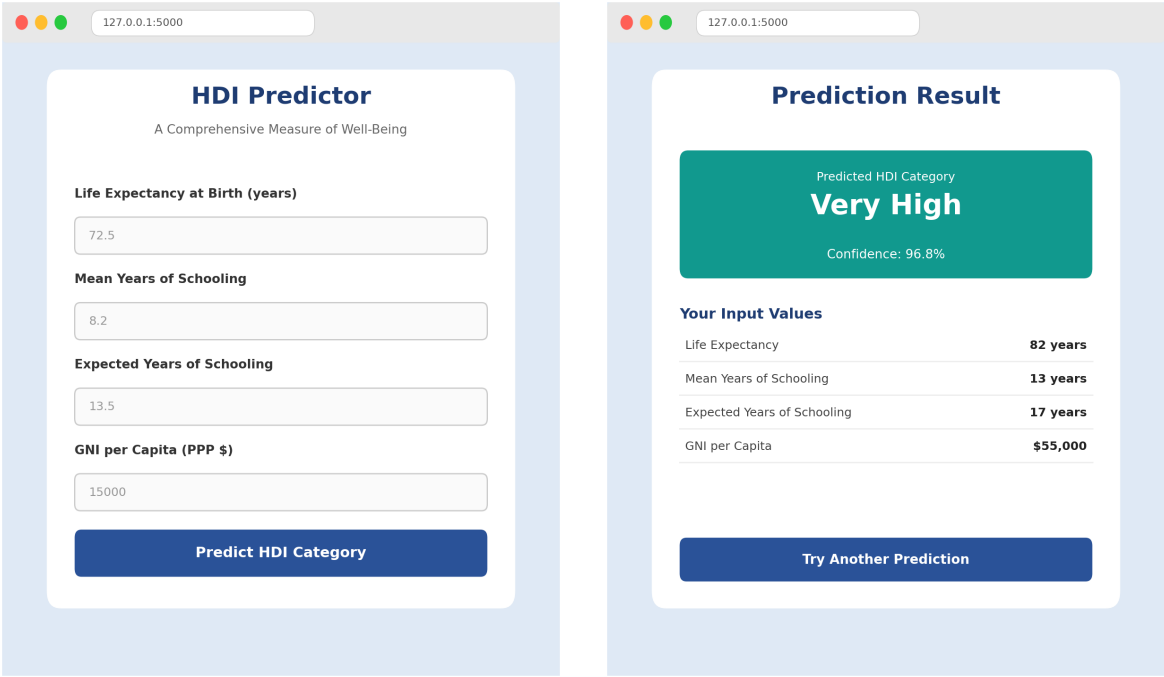


Figure 7: Input form (left) and prediction result page (right)

8. Project Structure

hdi_project/

- app.py - Flask application (Epic 8)
- train_model.py - Train/test split, model fit, save (Epic 5-7)
- preprocessing.py - Library imports, cleaning, encoding (Epic 2, 4)
- requirements.txt - Python dependencies (Epic 1)
- data/
 - ■■■ generate_dataset.py - Dataset generation (Epic 3)
 - ■■■ hdi_dataset.csv - 1,200-row generated dataset
- model/
 - ■■■ hdi_model.pkl, scaler.pkl, label_encoder.pkl
 - ■■■ confusion_matrix.png, feature_importance.png
- templates/ (index.html, result.html)
- static/ (style.css)

9. Conclusion

This project successfully demonstrates an end-to-end machine learning pipeline for predicting a country's Human Development Index category from four socio-economic indicators. The Random Forest model achieved 86.67% test accuracy, and its predictions for the three project scenarios (Very High, Medium, Low) matched the expected outcomes described in the brief. The model was deployed via a Flask web application, giving policymakers, researchers, and the public an accessible tool to explore how life expectancy, education, and income together shape overall human development.

GitHub Repository: <https://github.com/mlasya927/hdi-project>